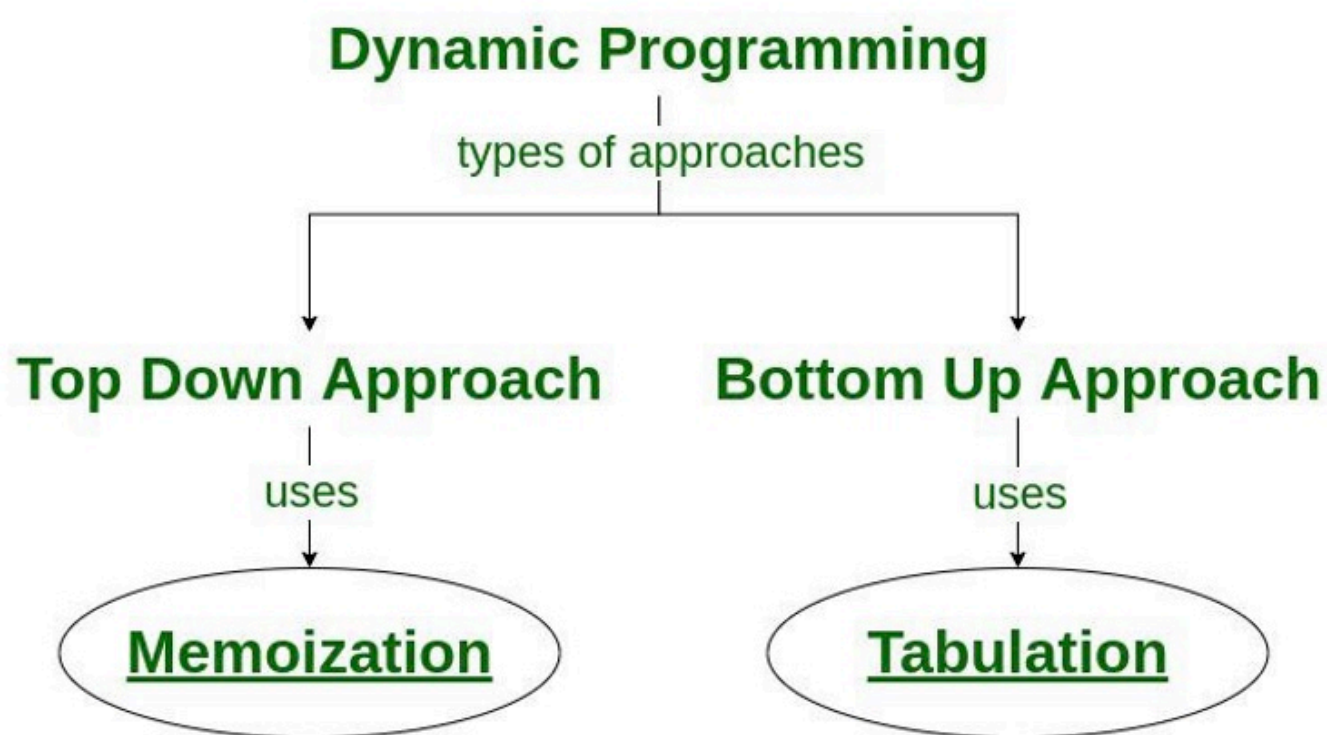


Dynamic Programming

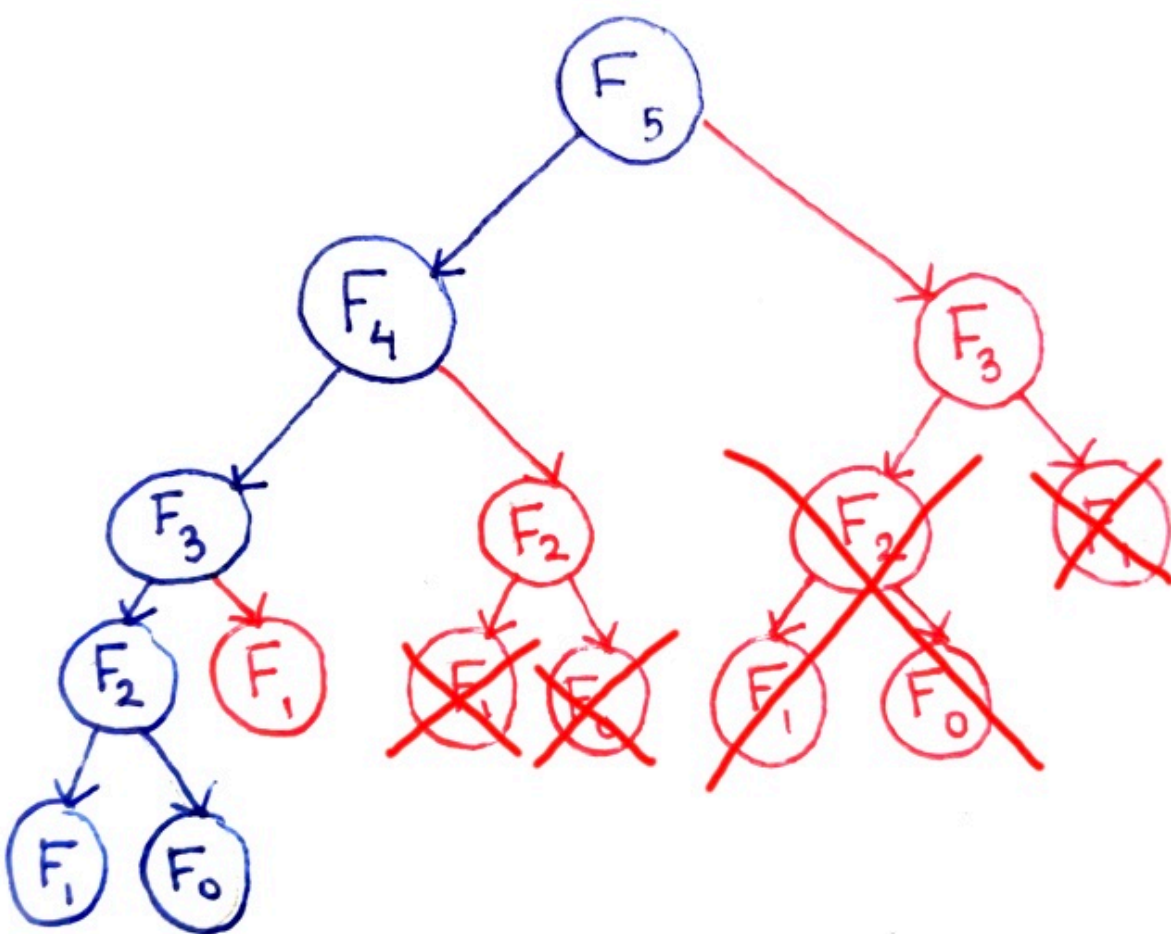
- Breaks down complex problems into overlapping sub-problems
- Uses solution of sub-problems to build solutions for larger ones.



- Recursive
- Stores results in a cache
- Avoids recomputation

- Iterative
- Builds a table from smallest subproblem

Fibonacci



- Goes from $O(2^n)$ to $O(n)$

0/1 knapsack

$$P = \{1, 2, 5, 6\} \quad m = 8$$

$$n = 4$$

$$W = \{2, 3, 4, 5\}$$

There are 2^n i.e. 2^4 solutions. Naive recursion will take $O(2^n)$. So we have to use DP.

Tabulation:

- Although capacity is 8, we won't consider it at once. We will slowly increase the capacity one by one.
- Similarly, we consider the objects one by one.
- y-axis represents no. of objects being included
x-axis represents current capacity.
Values in the cell represent profit.
- At i th row, we consider all objects before it as well.

			V									
				0	1	2	3	4	5	6	7	8
P_i	w_i		0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7	7
6	5	4	0	0	1	2	5	6	6	7	8	8

$$V[i, w] = \max \left\{ V[i-1, w], V[i-1, w - w[i]] + P[i] \right\}$$

• Now, to find solution:

• We find max Profit i.e. 8 & it appears only when

object 4 was included.

- Remaining profit is $8 - 6 = 2$.
- Check if profit = 2 is in 3rd row but also in 2nd. So, we got profit = 2 because of 2nd object & not 3rd.
- Remaining profit is $2 - 2 = 0$.
- Check if profit = 0 is in 1st row but also in 0th. So, we don't include it.

$$\begin{bmatrix} 0 & 1 & 0 & 1 \\ x_1 & x_2 & x_3 & x_4 \end{bmatrix}$$

Tabulation

```
def knapsack_bottom_up(weights, values, W):
    n = len(weights)
    # Create DP table initialized to 0
    dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(1, W + 1):
            # Check if item fits in current capacity w
            if weights[i-1] <= w:
                # Max of (Exclude item, Include item)
                dp[i][w] = max(dp[i-1][w], values[i-1] + dp[i-1][w - weights[i-1]])
            else:
                # Item too heavy, must exclude
                dp[i][w] = dp[i-1][w]

    return dp[n][W]
```

Rod Cutting Problem

$$P = \{1, 5, 8, 9\} \quad L = 4$$
$$l = \{1, 2, 3, 4\} \quad n = 4$$

Memoization

$$\text{memo}[L] = -1, \text{cuts}[L+1] = -1$$

if varying cuts use a dict:

$$\text{allowed_cuts} = \{1, 3, 9\}$$
$$\text{Profits} = \begin{cases} 1: 1, \\ 3: 8, \\ 9: 25 \end{cases}$$

for cut in allowed_cuts:

int cutRod(int L) {

if cut ≤ L:

if n == 0 return 0;

if memo[L] ≠ -1 return memo[L];

q = -inf
for i to L:

tmp = p[i] + cutRod(L-i)

if (tmp > q)

q = tmp

cuts[L] = i

memo[i] = q

return q;

}

cutRod(4)

i=1

i=2

i=4

i=3

5 + cutRod(2)

8 + cutRod(1)

9 + cutRod(0)

1 + cutRod(3)

i=1

i=2

i=3

1 + cutRod(2)

i=1

i=2

q = -inf

5 + cutRod(1)

8 + cutRod(0)

1 + cutRod(1)

5 + cutRod(0)

1 + cutRod(0)

memo [0 1 5 8 10] cuts [0 1 2 3 2]

cuts[4] = 2

cuts[4-2] = 2

Tabulation:

cutRod(int p[], int L) {

int dp = new int[L+1]

int cuts = new int[L+1]

for i = 1 → L:

for j = 0 → i

if (p[j] + dp[i-j-1] > dp[i])

dp[i] = p[j] + dp[i-j-1]

cuts[i] = j + 1

}

Unbounded knapsack

0 1 2 3 4 5
0 1 5 8 9 10 n=5

j → rod len

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	2	3	4	5
2	0	1	5	6	10	11
3	0	1	5	8	10	13
4	0	1	5	8	10	13
5	0	1	5	8	10	13

cut allowed ← i

2-2=0

5-3=2

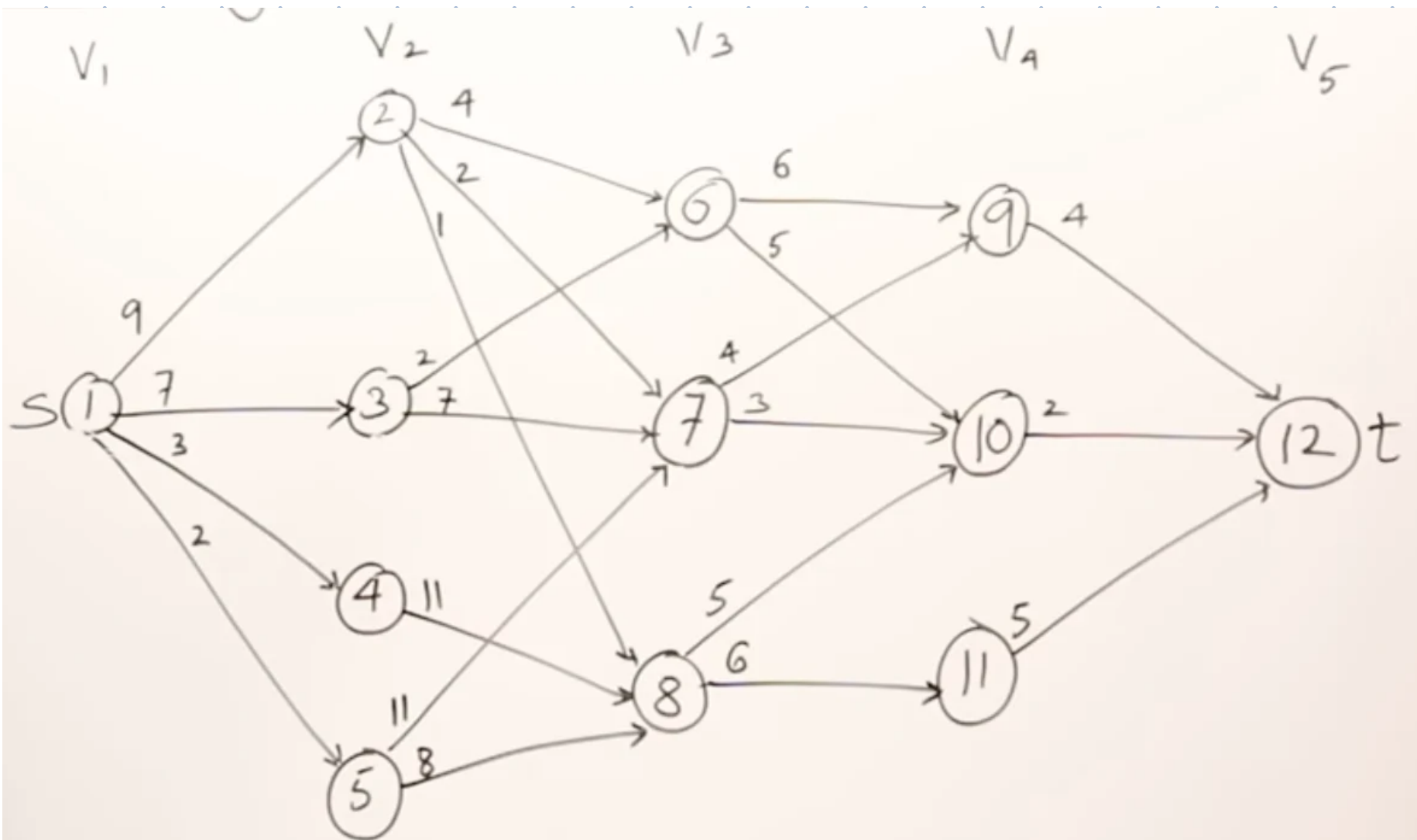
make cuts of len 2 & 3

13-8=5

5-5=0

Multi-stage Graph

- It is a directed graph divided into ordered stages.
- Edges exist only from stage i to stage $i+1$.
- Structure of multi-stage graph:
 - Nodes are grouped into stages
 - Start node in stage 1
 - End node in final stage
 - Weighted Edges between consecutive stages
- At each stage, we have to take a decision, so principle of optimality holds.



V	1	2	3	4	5	6	7	8	9	10	11	12
Cost	16	7	9	18	15	7	5	7	4	2	5	0
d	2/3	7	6	8	8	10	10	10	12	12	12	

vertex that gives min cost

Stage 5

cost(5, 12) = 0

Stage 4

cost(4, 9) = 4

stage

$$\text{cost}(4, 10) = 2$$

$$\text{cost}(4, 11) = 5$$

Stage 3

$$\begin{aligned} \text{cost}(3, 6) &= \min \left(C_{6,9} + \overbrace{\text{cost}(4, 9)}^{\text{from table}}, C_{6,10} + \overbrace{\text{cost}(4, 10)}^{\text{from table}} \right) \\ &= \min(6 + 4, 5 + 2) \end{aligned}$$

$$= 7$$

$$\begin{aligned} \text{cost}(3, 7) &= \min \left(C_{7,9} + \overbrace{\text{cost}(4, 9)}^{\text{from table}}, C_{7,10} + \overbrace{\text{cost}(4, 10)}^{\text{from table}} \right) \\ &= \min(4 + 4, 3 + 2) \end{aligned}$$

$$= 5$$

$$\begin{aligned} \text{cost}(3, 8) &= \min \left(C_{8,10} + \overbrace{\text{cost}(4, 10)}^{\text{from table}}, C_{8,11} + \overbrace{\text{cost}(4, 11)}^{\text{from table}} \right) \\ &= \min(5 + 2, 6 + 5) \end{aligned}$$

$$= 7$$

For stage 2:

$$\begin{aligned} \text{cost}(2, 2) &= \min \left(C_{2,6} + \text{cost}(3, 6), C_{2,7} + \text{cost}(3, 7), C_{2,8} + \text{cost}(3, 8) \right) \\ &= \min(4 + 7, 2 + 5, 1 + 7) \end{aligned}$$

$$= 7$$

$$\begin{aligned} \text{cost}(2, 3) &= \min \left(C_{3,6} + \text{cost}(3, 6), C_{3,7} + \text{cost}(3, 7) \right) \\ &= \min(2 + 7, 7 + 5) \end{aligned}$$

$$= 9$$

$$\begin{aligned} \text{cost}(2, 4) &= \min \left(C_{4,8} + \text{cost}(3, 8) \right) \\ &= \min(11 + 7) \end{aligned}$$

$$= 18$$

$$\begin{aligned} \text{cost}(2, 5) &= \min \left(C_{5,8} + \text{cost}(3, 8), C_{5,7} + \text{cost}(3, 7) \right) \\ &= \min (8 + 7, 11 + 5) \\ &= 15 \end{aligned}$$

For stage 1

$$\begin{aligned} \text{cost}(1, 1) &= \min \left[C_{1,2} + \text{cost}(2, 2), C_{1,3} + \text{cost}(2, 3), \right. \\ &\quad \left. C_{1,4} + \text{cost}(2, 4), C_{1,5} + \text{cost}(2, 5) \right] \\ &= \min [9 + 7, 7 + 9, 3 + 18, 2 + 15] \\ &= \min [16, 16, 21, 17] \\ &= 16 \end{aligned}$$

Formula:

$$\text{cost}(\underset{\substack{\downarrow \\ \text{stage no}}}{i}, \overset{\substack{\nearrow \\ \text{vertex}}}{j}) = \min_l C_{j,l} + \text{cost}(i+1, l)$$

Now, we use the table to find the path.

- As we move forward from source to destination, this method is called forward method.

$$d(\underset{\substack{\downarrow \\ \text{stage}}}{1}, \overset{\substack{\nearrow \\ \text{vertex}}}{1}) = 2 \quad \rightarrow \text{vertex in stage + 1}$$

Another path:

$$d(1, 1) = 3$$

$$d(2, 2) = 7$$

$$d(2, 3) = 6$$

$$d(3, 7) = 10$$

$$d(3, 6) = 10$$

$$d(4, 10) = 12$$

$$d(4, 10) = 12$$

Matrix Chain Multiplication



$$A \times B = C$$

$$\begin{bmatrix} \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix} \times \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} = \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}$$

Dimensions: $5 \times 4 \quad 4 \times 3$
must match

$$B \times A$$

$4 \times 3 \quad 5 \times 4$
not possible

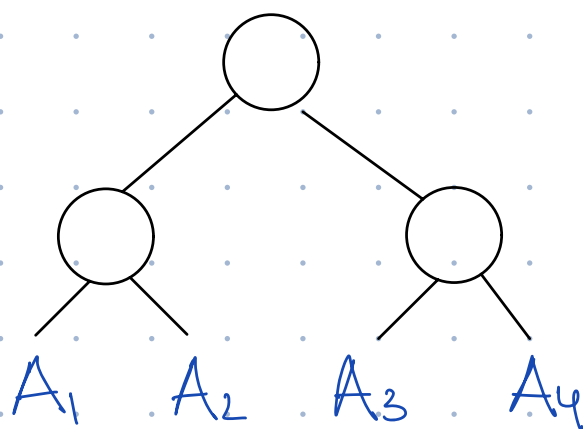
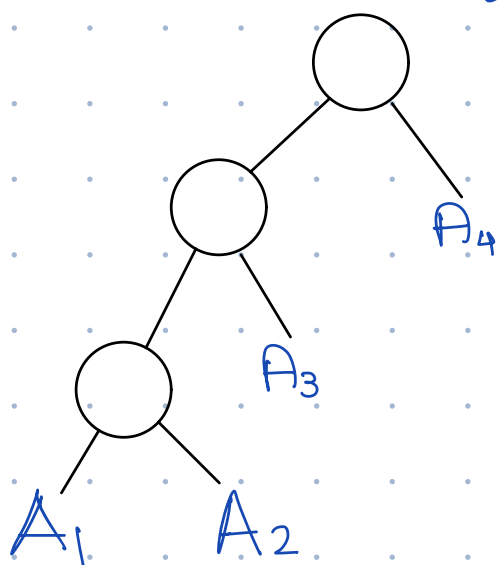
Total multiplication to compute $C = 5 \times 3 \times 4 = 60$

In Matrix chain multiplication, we have to do it pair-by-pair, so, optimization is to choose pairs such that total multiplications is minimal.

Possible Pairs:

$$\{ [(A_1 \cdot A_2) \cdot A_3] \} \cdot A_4$$

$$(A_1 \cdot A_2) \cdot (A_3 \cdot A_4)$$



Both have 3 nodes (multiplications)

So, with 3 nodes, we have combinations

$$T(n) = \frac{2n C n}{n+1}$$

$$= \frac{2 \times 3 C 3}{4}$$

$$= 5$$

We can either try all 5 OR use DP.

Dynamic Programming

$$A_1 \cdot A_2 \cdot A_3 \cdot A_4$$

$5 \times 4 \quad 4 \times 6 \quad 6 \times 2 \quad 2 \times 7$

Tabulation Method:

So, we start with smaller values & start filling up the table

m	1	2	3	4
1	0	120	88	156
2		0	48	104
3			0	84
4				0

S	1	2	3	4
1		1	1	3
2			2	3
3				3
4				

For one matrix:

$$m[1,1] = 0$$

A_1

$$m[3,3] = 0$$

A_3

$$m[2,2] = 0$$

A_2

$$m[4,4] = 0$$

A_4

Nothing is being multiplied. So, cost is 0.

For two matrices

$$m[1,2] = 5 \times 4 \times 6$$

$$A_1 \cdot A_2 = 120$$

$$5 \times 4 \quad 4 \times 6$$

$$m[2,3] = 4 \times 6 \times 2$$

$$= 48$$

$$A_2 \cdot A_3$$

$$4 \times 6 \quad 6 \times 2$$

$$m[3,4] = 6 \times 2 \times 7$$

$$A_3 \cdot A_4 = 84$$

$$6 \times 2 \quad 2 \times 7$$

For three matrices

$$m[1,3] = 88$$

$$A_1 \cdot A_2 \cdot A_3$$

for this we have
two possibilities

$$A_1 \cdot (A_2 \cdot A_3)$$

$$5 \times 4 \quad 4 \times 6 \quad 6 \times 2$$

$$(A_1 \cdot A_2) \cdot A_3$$

$$5 \times 4 \quad 4 \times 6 \quad 6 \times 2$$

$$\min \left[\begin{array}{l} m[1,1] + m[2,3] + (5 \times 4 \times 2) \\ 0 + 48 + 40 \\ 88 \end{array} , \begin{array}{l} m[1,2] + m[3,3] + (5 \times 6 \times 2) \\ 120 + 0 + 60 \\ 180 \end{array} \right]$$

Here, as partitioning after first matrix gave min, so put
 $S[1,3] = 1$

$$m[2,4] = 104$$

$$A_2 \cdot (A_3 \cdot A_4)$$

$$4 \times 6 \quad 6 \times 2 \quad 2 \times 7$$

$$(A_2 \cdot A_3) \cdot A_4$$

$$4 \times 6 \quad 6 \times 2 \quad 2 \times 7$$

$$\min \left\{ \begin{array}{l} m[2,2] + m[3,4] + (4 \times 6 \times 7) \\ 0 + 84 + 168 \\ 252 \end{array} , \begin{array}{l} m[2,3] + m[4,4] + (4 \times 2 \times 7) \\ 48 + 0 + 56 \\ 104 \end{array} \right\}$$

Here, As partitioning after 3rd matrix has given
min result, so $S[2,4] = 3$

For four matrices:

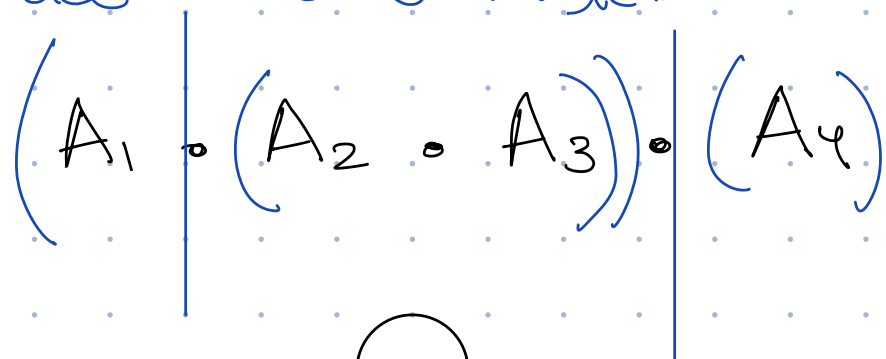
$$m[1,4] = \min \left\{ m[1,1] + m[2,4] + (5 \times 4 \times 7), \right.$$

$$\text{smallest} \leftarrow \left. \begin{aligned} &m[1,2] + m[3,4] + (5 \times 6 \times 7), \\ &m[1,3] + m[4,4] + (5 \times 2 \times 7) \end{aligned} \right\}$$

$\begin{matrix} 0 & + & 104 & + & 140 \\ 120 & + & 84 & + & 210 \\ 88 & + & 0 & + & 70 \end{matrix}$

$$m[i, j] = \min_{i \leq k < j} m[i, k] + m[k+1, j] + d_{i-1} * d_j * d_k$$

To get solution, we use the S table.



$$S[1, 4] = 3$$

$$S[1, 3] = 1$$

$$m = \frac{n(n+1)}{2} \text{ elements computed}$$

$$= n^2$$

we move from right-most diagonal to left-most till we reach main diagonal. So, we computed 1, 2, 3, 4 elements.

For each, we check for min configurations which is n .

So, total complexity

$$n^2 \times n$$

$$O(n^3)$$